WEAR-RAG: Weighted Evidence Aggregation for Multi-Hop Retrieval-Augmented Generation

First Author, *Student Member, IEEE,* Second Author, *Member, IEEE,* and Third Author, *Senior Member, IEEE*

Abstract

Retrieval-Augmented Generation (RAG) has become a practical way to ground large language model outputs in external evidence, yet many implementations still suffer from a familiar issue: retrieved context is not always equally useful, and weak passages can quietly dominate generation quality. This paper presents WEAR-RAG, a modular architecture that improves evidence quality before generation by combining semantic chunking, question decomposition, cross-encoder reranking, and weighted evidence aggregation. The key design choice is simple but effective: instead of forwarding top-k chunks as if they were equally trustworthy, WEAR-RAG computes a composite evidence score for each chunk using dense retrieval similarity, cross-encoder relevance, and information density. Chunks below a quality threshold are filtered, and only high-value evidence is passed to the language model.

We evaluate WEAR-RAG on 100 samples from HotpotQA (distractor validation split) against a baseline RAG system implemented in the same project. WEAR-RAG improves Exact Match from 0.0700 to 0.0800, token-level F1 from 0.2136 to 0.2339, ROUGE-L from 0.2123 to 0.2330, BLEU-1 from 0.1582 to 0.1733, and retrieval precision from 0.3480 to 0.3960, while preserving nearly identical Mean Reciprocal Rank (MRR). Beyond quantitative gains, the project provides reproducible tooling: a unified evaluator, visual diagnostics, unit tests, and a Flask-based interactive web interface with both production and lightweight mock modes.

The findings suggest that better evidence selection, not just better retrieval, is a meaningful lever for practical RAG quality in multi-hop question answering.

Index Terms

Retrieval-Augmented Generation, Multi-hop Question Answering, Evidence Aggregation, Reranking, HotpotQA, Explainable Retrieval.

I. Introduction

Large language models (LLMs) are remarkably strong at synthesis and fluent response generation, but fluency can hide factual fragility. In applied settings such as educational assistants, enterprise search copilots, legal document QA, and scientific summarization, confidence without grounding is expensive. Retrieval-Augmented Generation (RAG) has emerged as a practical compromise: keep generation power, but condition it on retrieved passages from a corpus [1]. In production, however, the quality bottleneck often moves from generation to context selection.

Manuscript prepared on April 1, 2026.

The authors are with the Department of Computer Science, Your Institute, Your City, India (e-mail: author1@institute.edu).

A common RAG pattern retrieves top-k chunks using dense embeddings and forwards them to the generator. This design is computationally attractive but semantically brittle for multi-hop tasks: some chunks are highly relevant, some are adjacent but noisy, and some are only weakly connected. Treating all retrieved chunks as equally reliable creates avoidable failure modes. The model may over-attend to low-signal passages, underuse key evidence, or produce diluted answers that are directionally correct but incomplete.

This paper presents WEAR-RAG (Weighted Evidence Aggregation for RAG), a system designed around one guiding principle: context should be curated by evidence value, not by retrieval rank alone. WEAR-RAG integrates four components:

1) Semantic chunking to preserve topic coherence within chunks.
2) Query decomposition to better support multi-hop questions.
3) Cross-encoder reranking for pairwise relevance refinement.
4) Weighted evidence aggregation to score, filter, and order context.

The architecture is implemented end-to-end in Python and includes evaluation, visualization, and a web interface. We intentionally keep the design modular to support ablations and practical deployment.

A. Motivation from Engineering Practice

The project was developed with a practical target: make a RAG system that can run locally, expose understandable evidence traces, and be tested quickly. During early iterations, we observed a repeated pattern: retrieval would often include one or two very strong passages mixed with multiple weak passages. If all were passed to the generator unchanged, answer quality became unstable. This motivated explicit evidence scoring and thresholding as first-class pipeline stages rather than post-hoc diagnostics.

B. Contributions

This work makes the following contributions:

1) **WEAR-RAG architecture:** A complete, modular retrieval-to-generation pipeline with weighted evidence curation.
2) **Composite evidence scoring:** A practical scoring function that combines dense similarity, cross-encoder relevance, and information density.
3) **Reproducible implementation:** Shared model registry, deterministic mock mode, evaluator, unit tests, and web endpoints.
4) **Empirical findings:** On HotpotQA-100, WEAR-RAG improves answer and retrieval quality over baseline RAG.

II. Related Work

A. Retrieval-Augmented Generation

RAG frameworks couple a retriever with a generator to reduce hallucination and increase controllability [1], [2]. Dense Passage Retrieval (DPR) and similar bi-encoder retrievers scale well but encode query and passage

independently [3]. This tradeoff supports speed but can miss subtle interactions that matter in hard multi-hop questions.

B. Cross-Encoder Reranking

Cross-encoders jointly process query-passage pairs and typically improve ranking quality at higher inference cost. Practical systems frequently use two-stage retrieval: bi-encoder for candidate recall, cross-encoder for precision [4]. WEAR-RAG follows this pattern but extends it with explicit evidence aggregation rather than selecting reranked top-k blindly.

C. Multi-Hop QA and Decomposition

HotpotQA [5] highlights the need for compositional reasoning over multiple supporting facts. Question decomposition has been explored as a way to break complex queries into answerable components, improving retrieval coverage in downstream reasoning systems. WEAR-RAG adopts this idea using a rule-based decomposer by default and supports LLM-based decomposition as an optional extension.

D. Evidence Selection and Grounded Generation

A recurring challenge in grounded generation is relevance concentration: not every retrieved chunk deserves prompt budget. Prior work has explored confidence-aware selection, citation-grounded prompting, and reranking-based compression. WEAR-RAG contributes a compact, interpretable scoring function oriented toward implementability in constrained local settings.

III. Methodology

A. Problem Definition

Given a question q and document set $D = \{d_1, d_2, ..., d_n\}$, predict answer a using evidence set $E \subseteq D$. The objective is to maximize answer quality while minimizing irrelevant context.

B. Pipeline Overview

WEAR-RAG executes the following stages:

1) **Chunking:** Split each document into semantically coherent chunks.
2) **Decomposition:** Convert q into sub-queries $Q = \{q_1, ..., q_m\}$ with $q \in Q$.
3) **Dense retrieval:** Retrieve candidate chunks per sub-query from FAISS.
4) **Reranking:** Re-score query-chunk pairs using a cross-encoder.
5) **Aggregation:** Compute weighted evidence scores, filter, and rank.
6) **Generation:** Build grounded prompt and generate answer with Ollama/Mistral.

C. Semantic Chunking

Each document is sentence-tokenized and embedded sentence-wise. Chunk boundaries are introduced when adjacent sentence similarity falls below threshold τ or when chunk size exceeds a maximum budget.

Let sentence embeddings be s_i. Adjacent similarity is

$$\text{sim}(i, i+1) = \frac{s_i^\top s_{i+1}}{\|s_i\|\|s_{i+1}\|}. \tag{1}$$

A new chunk starts when $\text{sim}(i, i+1) < \tau$ subject to minimum chunk size constraints.

D. Query Decomposition

The default decomposer applies lightweight heuristics for comparison, causal, and compound patterns. It always includes the original question to preserve intent. For comparative questions such as ”Why is A better than B?”, additional sub-queries target definitions, strengths, and limitations separately.

E. Two-Stage Retrieval

Stage 1: Dense retrieval. Candidate chunks are retrieved with BAAI/bge-small-en embeddings from a FAISS IndexFlatIP index over normalized vectors.

Stage 2: Cross-encoder reranking. Candidate pairs (q_i, c_j) are reranked with BAAI/bge-reranker-base. Raw logits are mapped via sigmoid to obtain interpretable relevance scores in $[0, 1]$.

F. Weighted Evidence Aggregation

This is the core WEAR-RAG contribution. For each candidate chunk c, define

$$S(c) = w_{sim} S_{sim}(c) + w_{rank} S_{rank}(c) + w_{dens} S_{dens}(c). \tag{2}$$

where:

- S_{sim} is dense retrieval similarity,
- S_{rank} is cross-encoder relevance,
- S_{dens} estimates information density,
- $w_{sim} + w_{rank} + w_{dens} = 1$.

Default settings are $w_{sim} = 0.5$, $w_{rank} = 0.4$, $w_{dens} = 0.1$. Chunks are retained only if $S(c) \geq \theta$ with threshold $\theta = 0.3$. Remaining chunks are sorted and optionally truncated by token budget.

G. Information Density Heuristic

S_{dens} is a bounded heuristic combining normalized chunk length, type-token diversity, and a lightweight named-entity/technical-term proxy. While simple, it rewards richer chunks and helps de-prioritize very short or repetitive text.

H. Generation Policy

The generation prompt contains labeled evidence blocks and strict instructions to answer only from provided evidence. If evidence is inadequate, the model is instructed to output ”I don’t know.” This policy limits unsupported synthesis and improves user trust.

IV. System Design and Implementation

A. Module Architecture

The implementation is organized into independent modules:

- *document_processor.py*: semantic chunking.
- *embeddings.py*: passage/query embedding logic.
- *vector_store.py*: FAISS indexing and retrieval.
- *query_decomposer.py*: rule-based and optional LLM decomposition.
- *reranker.py*: cross-encoder reranking.
- *evidence_aggregator.py*: scoring, filtering, budget trimming.
- *llm_generator.py*: Ollama and mock generators.
- *evaluator.py*: metrics and CSV output.
- *visualizer.py*: charts and score visual diagnostics.

B. Shared Model Registry

To avoid repeated model loading across experiments, WEAR-RAG uses a model registry that initializes embedding and reranking models once and reuses them across systems. This significantly reduces repeated initialization overhead and stabilizes evaluation runtime.

C. Baseline and Improved Pipelines

The codebase includes:

- **Baseline RAG**: fixed chunking, single retrieval, no reranker.
- **Improved RAG**: decomposition-aware diversity retrieval.
- **WEAR-RAG**: full pipeline with reranking and weighted aggregation.

The paper benchmarks Baseline RAG and WEAR-RAG to isolate end-to-end impact.

D. Web Application Layer

A Flask application provides endpoints for health checks, question answering, and file upload (.txt/.pdf). The interface supports both production inference and mock mode for low-resource demonstrations. This deployment layer was useful for human-in-the-loop error analysis and qualitative inspection of evidence rankings.

V. Experimental Setup

A. Dataset

Experiments use the HotpotQA distractor validation split [5]. We evaluate on 100 samples to balance reproducibility and local compute constraints.

B. Metrics

We report six metrics:

1) Exact Match (EM)
2) Token-level F1
3) ROUGE-L
4) BLEU-1
5) Mean Reciprocal Rank (MRR)
6) Retrieval Precision

C. Model and Hyperparameter Settings

- Embedding model: BAAI/bge-small-en
- Reranker model: BAAI/bge-reranker-base
- Generator: Ollama Mistral (or mock mode)
- Retrieval: top_k_retrieval = 20
- Reranked keep: top_k_rerank = 5
- Aggregation threshold: 0.3
- LLM context window: 4096

D. Reproducible Commands

Typical run command:

```
python main.py --mode evaluate --samples 100
```
(3)

Mock run (no Ollama dependency):

```
python main.py --mode evaluate --samples 100 --mock
```
(4)

VI. Results

A. Main Comparison

Table I summarizes results from project-generated CSV files.

B. Delta Analysis

The nearly unchanged MRR with improved retrieval precision is informative: both systems often surface at least one relevant item early, but WEAR-RAG better curates the final evidence set that drives generation quality.

TABLE I
PERFORMANCE ON HOTPOTQA (100 SAMPLES)

System	EM	F1	ROUGE-L	BLEU-1	MRR	Retrieval Precision
Baseline RAG	0.0700	0.2136	0.2123	0.1582	0.9558	0.3480
WEAR-RAG	**0.0800**	**0.2339**	**0.2330**	**0.1733**	0.9550	**0.3960**

TABLE II
ABSOLUTE AND RELATIVE GAINS OF WEAR-RAG OVER BASELINE

Metric	Absolute Delta	Relative Delta
EM	+0.0100	+14.29%
F1	+0.0203	+9.50%
ROUGE-L	+0.0207	+9.75%
BLEU-1	+0.0151	+9.54%
MRR	-0.0008	-0.08%
Retrieval Precision	+0.0480	+13.79%

C. Qualitative Observations

In manual inspections, WEAR-RAG answers were generally less verbose and more grounded. The system showed better behavior on comparison-style questions where decomposition separated concept definitions from differential reasoning. Failure cases still occurred when supporting evidence was sparse or when distractor passages were semantically similar to the true supporting context.

D. Visualization Artifacts

The project exports comparison charts and metric breakdowns, allowing quick diagnostic checks during iteration. Evidence-level score tables are particularly useful in debugging threshold settings and validating whether reranker influence is calibrated appropriately.

VII. DISCUSSION

A. Why the Gains Matter

Absolute improvements may look modest at first glance, but the consistency across multiple answer metrics indicates meaningful system-level improvement, not random fluctuation in a single score. In practical deployments, even single-digit relative gains can reduce downstream correction cost, especially when QA output is reviewed by humans.

B. Engineering Tradeoffs

WEAR-RAG increases pipeline complexity compared with baseline RAG. Additional compute is consumed by decomposition and reranking, and threshold tuning introduces another control dimension. However, modularization keeps this complexity manageable. Components can be toggled for latency-sensitive deployments.

C. Humanized System Behavior

The project objective was not only metric gains but also more human-aligned behavior: answers should be concise, evidence-backed, and transparent in source attribution. Evidence ranking displays and source-level traces improved interpretability during iterative testing.

D. Where WEAR-RAG Helps Most

From qualitative review logs and evidence traces, WEAR-RAG appears particularly helpful in three situations:

1) **Comparison-heavy prompts:** Questions asking why one concept is better than another are naturally split into sub-problems, reducing retrieval collapse into a single perspective.
2) **Distractor-rich contexts:** Cross-encoder reranking lowers the influence of lexical look-alikes that often survive dense retrieval.
3) **Context-budget pressure:** Weighted filtering prevents low-value chunks from consuming prompt budget.

E. Where Gains Are Smaller

The performance delta tends to shrink when baseline retrieval is already highly aligned with the gold supporting facts. In such cases, stronger curation helps less because the input set is already clean. This explains why MRR remains almost unchanged while answer metrics improve: the first relevant hit is often present in both systems, but overall context composition still differs.

VIII. Complexity and Efficiency Analysis

This section summarizes practical cost characteristics observed during development.

A. Asymptotic View

Let N be number of chunks, m number of sub-queries, and k retrieval depth per sub-query.

- Dense retrieval in FAISS IndexFlatIP is efficient in practice for moderate N, with exact search and predictable behavior.
- Reranking cost scales roughly with $m \cdot k$ query-chunk pairs.
- Aggregation is linear in candidate count and negligible compared with reranking.

The main added cost versus baseline is cross-encoder reranking. This is a deliberate design choice: WEAR-RAG spends additional compute to reduce evidence noise before generation.

B. Memory and Runtime Notes

A practical engineering concern in local experiments is repeated model initialization. The shared model registry pattern amortizes this cost. During evaluation, each sample uses a fresh temporary vector index to avoid leakage across questions, while still reusing loaded embedding and reranking models.

C. Latency Tradeoff Perspective

WEAR-RAG is not intended to be the absolute fastest RAG variant. Instead, it targets an efficient quality-latency balance for settings where answer correctness and evidence transparency are more important than millisecond-level speed.

IX. Error Taxonomy and Qualitative Failure Modes

To make the system analysis actionable, we tracked failures by type rather than only by metric drops.

A. Failure Type A: Retrieval Miss

Sometimes supporting facts do not appear in top retrieval candidates. In this case, reranking and aggregation cannot recover missing evidence. The likely remedy is stronger retriever training or larger candidate depth.

B. Failure Type B: Decomposition Drift

Rule-based decomposition can occasionally generate sub-queries that are syntactically valid but semantically broad. This introduces off-target retrieval and slightly noisier aggregation.

C. Failure Type C: Over-Compression

When thresholding is too aggressive, the system may remove useful but medium-scoring context. This can produce concise yet incomplete answers.

D. Failure Type D: Generator Under-Specification

Even with curated evidence, the generator may produce under-explained answers if prompt space is tight or question framing is ambiguous. This is a prompt-policy issue rather than a retrieval issue.

E. Human-Centered Debugging Workflow

A practical debugging loop emerged:

1) Inspect sub-queries for semantic drift.
2) Inspect top reranked evidence and component scores.
3) Compare retained versus filtered chunks.
4) Review final prompt context and answer behavior.

This workflow reduced blind hyperparameter tuning and helped isolate whether errors came from retrieval, aggregation, or generation.

X. Deployment and Productization Considerations

Although this project is research-oriented, deployment practicality was a first-class goal.

A. Service Interface

The Flask API design keeps the system composable with external frontends:

- upload endpoint for raw document ingestion,
- question endpoint returning answer and evidence,
- health endpoint for local model readiness checks.

This API shape supports both direct user interfaces and integration in larger orchestration systems.

B. Operational Modes

Two modes are useful in practice:

1) **Production mode:** full WEAR-RAG with dense retrieval, reranking, and local LLM.
2) **Mock mode:** deterministic lightweight mode for demos, frontend tests, and CI checks.

Mock mode was especially valuable for separating UI and API validation from heavy model dependencies.

C. Safety and Trust UX

Evidence-first response design improves user trust only if surfaced clearly. WEAR-RAG returns evidence blocks with source IDs and score breakdowns, enabling lightweight provenance inspection. In high-stakes settings, this should be combined with stricter citation rendering and user-facing uncertainty indicators.

XI. Reproducibility Protocol

To support repeatability, we recommend the following protocol for future runs:

1) Fix random seeds for all sampling and model components where feasible.
2) Freeze package versions in a complete requirements lock file.
3) Record model identifiers and local runtime settings.
4) Run baseline and WEAR-RAG on identical sample IDs.
5) Export per-sample CSV outputs and archive plots.
6) Report both aggregate metrics and error-category counts.

A. Suggested Reporting Template

A robust experiment card should include dataset split, sample count, runtime hardware, retriever and reranker models, prompt settings, and confidence intervals for each metric.

B. Versioned Artifact Bundle

For paper reproducibility, a versioned artifact bundle should contain:

- source code snapshot,
- exact config files,
- result CSV files,
- generated figures,
- environment manifest.

XII. Expanded Threats to Validity

Beyond dataset and model constraints, two practical factors affect interpretation.

A. Prompt-Specific Effects

Prompt wording can shift generation behavior significantly, even with fixed evidence. Reported gains therefore reflect both retrieval quality and prompt policy interactions.

B. Infrastructure Variance

Local CPU/GPU differences influence throughput and occasionally timeout behavior in end-to-end evaluations. Quality metrics are less sensitive than runtime metrics, but deployment conclusions should still account for environment differences.

XIII. Ablation Plan and Expected Effects

The current benchmark compares baseline against full WEAR-RAG. The architecture naturally supports ablations:

1) Remove decomposition: likely lower multi-hop coverage.
2) Remove reranker: likely noisier evidence ordering.
3) Remove density term: likely weaker distinction between rich and shallow chunks.
4) Set threshold to zero: likely more context noise.
5) Replace semantic chunking with fixed chunking: likely coherence loss.

A practical next experiment is to run all ablations on the same 100-sample slice for direct comparability, then expand to larger sample sizes for confidence intervals.

XIV. Limitations and Threats to Validity

A. Dataset Scope

Results are limited to HotpotQA distractor validation and a 100-sample subset. Generalization to other domains and question types requires broader evaluation.

B. Statistical Testing

This draft reports aggregate means without significance tests. Bootstrap confidence intervals or paired significance tests should be included in publication-grade finalization.

C. Dependency Reproducibility

The root dependency list in the project should be expanded and pinned to guarantee one-command reproducibility across environments.

D. Model Variability

Generation quality can vary with local Ollama model builds, hardware constraints, and runtime settings. Mock mode mitigates this for unit tests but not for end-task quality evaluation.

E. Scope of Human Evaluation

This draft focuses on automated metrics and engineering diagnostics. A fuller publication should include human evaluation on answer usefulness, faithfulness, and readability.

XV. Future Research Directions

WEAR-RAG opens several promising paths:

1) **Learned aggregation:** Replace fixed weights with learned calibration from supervision or preference signals.
2) **Adaptive thresholds:** Use query-aware thresholding instead of static global values.
3) **Domain adaptation:** Fine-tune retriever and reranker on target domains (biomedical, legal, finance).
4) **Citation-grounded generation:** Force sentence-level evidence attribution in outputs.
5) **Long-context integration:** Combine WEAR-RAG with long-context models and hierarchical retrieval.

XVI. Practical Lessons Learned

This project yielded several practical lessons that may help teams building real-world RAG systems:

1) Better chunking and better reranking are necessary but not sufficient; explicit evidence filtering still matters.
2) Diagnostics should be built into the pipeline from day one. Score traces and per-sample CSVs speed up debugging dramatically.
3) Separation of concerns (retrieval, aggregation, generation) makes iteration safer and more measurable.
4) Lightweight mock modes are not optional in engineering-heavy workflows; they keep demos and CI stable.

XVII. Ethical and Responsible Use Considerations

Grounded generation systems are often perceived as inherently trustworthy. This perception can be misleading if retrieval itself is biased or incomplete. WEAR-RAG reduces some risk by surfacing explicit evidence and encouraging abstention when context is insufficient, but it does not solve source quality bias. Deployment should include provenance checks, user-visible confidence cues, and domain-specific review pathways for high-stakes use cases.

XVIII. Conclusion

WEAR-RAG demonstrates that improving evidence curation between retrieval and generation can produce reliable quality gains in multi-hop QA. The method is intentionally practical: all components are transparent, modular, and deployable on local infrastructure. On HotpotQA-100, WEAR-RAG consistently outperforms baseline RAG in EM, F1, ROUGE-L, BLEU-1, and retrieval precision while preserving retrieval rank behavior (MRR). These results support a broader lesson: in RAG systems, context quality is an optimization target in its own right.

Appendix A
Implementation Notes

A. Shared Model Loading

A model registry pattern is used to load heavy embedding and reranking models once and share across pipeline instances, improving evaluation throughput and reducing startup overhead.

B. Web API Endpoints

The Flask app includes:

- `/api/health`: model and server readiness check.
- `/api/upload`: TXT/PDF parsing for document ingestion.
- `/api/ask`: end-to-end question answering with evidence output.

C. Testing

Unit tests cover sentence splitting, chunk construction, metric correctness, aggregation behavior, token-budget trimming, and decomposition constraints.

D. Configuration Snapshot

Default settings used in this work:

- Embedding model: BAAI/bge-small-en
- Reranker model: BAAI/bge-reranker-base
- top_k_retrieval: 20
- top_k_rerank: 5
- score threshold: 0.3
- evidence weights: (0.5, 0.4, 0.1)

APPENDIX B
EXTENDED MODULE WALKTHROUGH

A. Document Processor

The semantic chunker balances coherence and size constraints. Adjacent sentence similarity decides topic boundaries, while overlap preserves continuity near split points.

B. Embedding Engine

The embedding layer prefixes instructions for passages and queries and normalizes vectors to ensure inner product in FAISS corresponds to cosine similarity.

C. Vector Store

The FAISS-backed store maps integer IDs to chunk metadata, supports retrieval for single and multi-query workflows, and can persist to disk for reuse.

D. Reranker

Cross-encoder reranking re-evaluates candidates jointly with the query, then deduplicates by chunk ID while keeping best-scoring instances.

E. Evidence Aggregator

Aggregation computes component and composite scores, applies thresholding, and trims by token budget. This makes context assembly both controllable and interpretable.

F. Generator

The generator is instructed to answer from evidence only and abstain when evidence is insufficient. This policy is central to reducing unsupported claims.

Appendix C
Pseudo-code for WEAR-RAG

Input: question q, documents D

Output: answer a, ordered evidence set E

1) Chunk documents semantically: $C \leftarrow \text{SemanticChunk}(D)$.
2) Build retrieval index over C.
3) Decompose query: $Q \leftarrow \text{Decompose}(q)$ with $q \in Q$.
4) For each sub-query $q_i \in Q$, retrieve candidates R_i (top-20).
5) Rerank and deduplicate candidates across all R_i.
6) For each chunk c, compute evidence score using Eq. (2).
7) Filter by threshold: keep chunks with $S(c) \geq \theta$.
8) Sort by $S(c)$ and trim by token budget.
9) Generate answer from question plus curated evidence context.

Appendix D
Extended Error Analysis Checklist

1) Was at least one supporting fact retrieved in top-20?
2) Did decomposition create useful sub-queries or noisy expansions?
3) Did reranker overvalue lexical overlap over semantic relevance?
4) Were low-density chunks still admitted due to high similarity?
5) Did generation abstain appropriately when evidence was inadequate?

Appendix E
Case Study Snapshots

A. Case 1: Comparative Multi-hop Question

Question form: ”Why is X better than Y for task Z?”

Observed behavior: decomposition generated targeted sub-queries for each concept and comparative aspects. Aggregation promoted one high-quality evidence chunk per conceptual branch, leading to a more complete final answer than baseline retrieval.

B. Case 2: Distractor-Dense Context

Question form: entity relation with many lexically similar distractors.

Observed behavior: reranker reduced lexical traps, and score thresholding removed short, repetitive snippets. Final generation showed improved factual focus.

C. Case 3: Sparse Evidence Scenario

Question form: under-specified or weakly represented in corpus.

Observed behavior: both systems struggled, but WEAR-RAG more often produced constrained or abstaining responses rather than overconfident narratives.

APPENDIX F
EXPANDED REPRODUCIBILITY CHECKLIST

1) Confirm dataset variant and split exactly match reported setup.
2) Verify model names and versions for embedding/reranking/generation.
3) Ensure equal sample list across compared systems.
4) Store raw predictions, not just aggregate metrics.
5) Track retrieval IDs and evidence scores per sample.
6) Re-run at least three times for stability checks where stochasticity exists.
7) Report runtime environment (OS, Python version, hardware).
8) Archive all generated plots and logs with timestamps.

ACKNOWLEDGMENT

The authors thank the project reviewers and peers who provided iterative feedback on retrieval diagnostics, evaluation clarity, and interface usability.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "Realm: Retrieval-augmented language model pre-training," in *International Conference on Machine Learning*, 2020, pp. 3929–3938.

[3] V. Karpukhin, B. Oguz, S. Min *et al.*, "Dense passage retrieval for open-domain question answering," in *Proceedings of EMNLP*, 2020, pp. 6769–6781.

[4] R. Nogueira and K. Cho, "Passage re-ranking with bert," in *arXiv preprint arXiv:1901.04085*, 2019.

[5] Z. Yang, P. Qi, S. Zhang *et al.*, "Hotpotqa: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of EMNLP*, 2018, pp. 2369–2380.